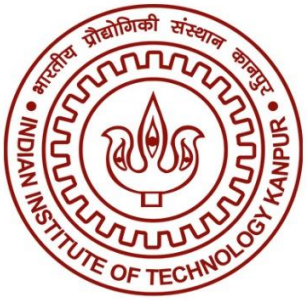


Indian Institute of Technology, Kanpur

Department of Civil Engineering

CE432: Geographical Information System 2025-26/II

Project: Metadata Integrity and Tampering Detection in GIS Data

Group 3

Instructor:

- Dr. Salil Goel
- Maj. Gen (Dr.) B. Nagarajan

Group Members:

- Ankit Kumar (230152)
- Kanika Sharma (230524)
- Shivesh Shukla (230978)

Date of Submission: 19-04-2026

Abstract

GIS metadata contains critical information such as coordinate system, scale, accuracy, source, and timestamp. Any unauthorized modification of metadata, without altering spatial data can lead to incorrect spatial analysis, policy decisions and data misuse. Currently, there is no dedicated tool specifically for detecting metadata tampering in QGIS or GIS files. GAIT (Geospatial Analysis Integrity Tool) developed by the National Geospatial-Intelligence Agency (NGA), validates geospatial data against a data model, checking geometry, feature codes, attribute values and domains, and metadata, but it's focused on schema conformance, not forensic tampering detection. To address this gap, this project develops a metadata verifier, which is a QGIS plugin for detecting tampering in metadata of Geospatial layers. For detecting metadata tampering using a multi-layered approach, a cryptographic module employs HMAC-SHA256 signatures with keys derived via PBKDF2, ensuring secure verification without storing sensitive keys. In addition, an NLP-based semantic checker evaluates internal consistency of metadata using rule-based validation, independent of any baseline and a machine learning–based detector further enhances detection by identifying anomalous metadata patterns.

The system was evaluated on a dataset of 21 layers with controlled tampering scenarios. Results demonstrate high precision and specificity, along with efficient verification performance. The proposed approach combines deterministic, rule-based, and learning-based techniques to provide a lightweight and reliable solution for ensuring geospatial metadata integrity in practical GIS workflows.

Introduction

Geographic Information Systems (GIS) are powerful frameworks that capture, analyse, and visualize spatial data to answer the critical question of what is happening and exactly where, transforming raw geographic information into actionable decisions across urban planning, disaster management, environmental monitoring, precision agriculture, and defence [9]. In urban contexts, GIS guides city design and infrastructure siting, in disasters, it drives real-time evacuation routing and damage assessment, environmentally, it tracks deforestation and carbon stocks within days of change, in agriculture, it prescribes variable inputs across fields to maximize yield, and in defence, it underpins terrain analysis and border surveillance. Accuracy in GIS is non-negotiable, a 50-metre error in a flood boundary can endanger thousands, while a misclassified coordinate in a defence system can trigger diplomatic crises, making precision the foundation of trust in every GIS-driven decision. As open satellite data, cloud computing, and AI continue to evolve, GIS has grown from a specialist mapping tool into essential decision-making infrastructure, increasingly embedded in how governments, industries, and organizations across the world operate and respond.

Metadata in GIS is the identity card of any spatial dataset, recording critical details like the coordinate reference system, spatial extent, projection, author, creation date, and data quality information, all formally standardized under ISO 19115 [1], the international standard that ensures datasets are consistently described and discoverable across organizations. The coordinate reference system alone determines how every point is mathematically placed on

the Earth's surface. ISO 19115 exists precisely to define the common language through which GIS data can be correctly interpreted and trusted across systems worldwide.

Metadata can be modified intentionally or accidentally, without leaving any visible trace on the map, making tampering one of the most silent threats in geospatial systems. The map still looks normal, but the underlying properties are wrong, a quietly changed CRS field, for instance, causes the entire layer to shift geographically by hundreds of metres with no visual warning. In critical applications like flood mapping or land records, this can place evacuation boundaries in the wrong location or misattribute land ownership, leading to seriously wrong decisions. And as GIS data is increasingly shared across platforms and organizations, the risk of undetected tampering only grows, which is why this project investigates these threats and proposes integrity verification mechanisms to ensure GIS datasets can be fully trusted.

Existing GIS tools like QGIS are built primarily for visualization, spatial analysis, and data editing, not for integrity verification. There is no built-in mechanism in QGIS or most standard GIS platforms to detect whether metadata has been altered after a dataset was created, leaving a significant gap in how geospatial data is trusted and validated. Cryptographic hashing, a well-established technique used across software distribution, file verification, and blockchain systems to detect any change in data, has not yet been applied in a practical, accessible form to GIS metadata verification, and this project addresses exactly that gap [5].

This project presents a QGIS plugin, Metadata Verifier, designed to ensure the integrity of GIS layer metadata and detect unauthorized modifications. The system combines cryptographic SHA-256 hash-based verification using PBKDF2 derived keys, an NLP based semantic checker for internal consistency analysis, and a machine learning–based detector for identifying anomalous patterns. It supports secure hash generation for data owners and multi-layered verification for end users. The plugin was evaluated on a dataset of GIS layers under controlled tampering scenarios, demonstrating high reliability and efficiency across different verification approaches.

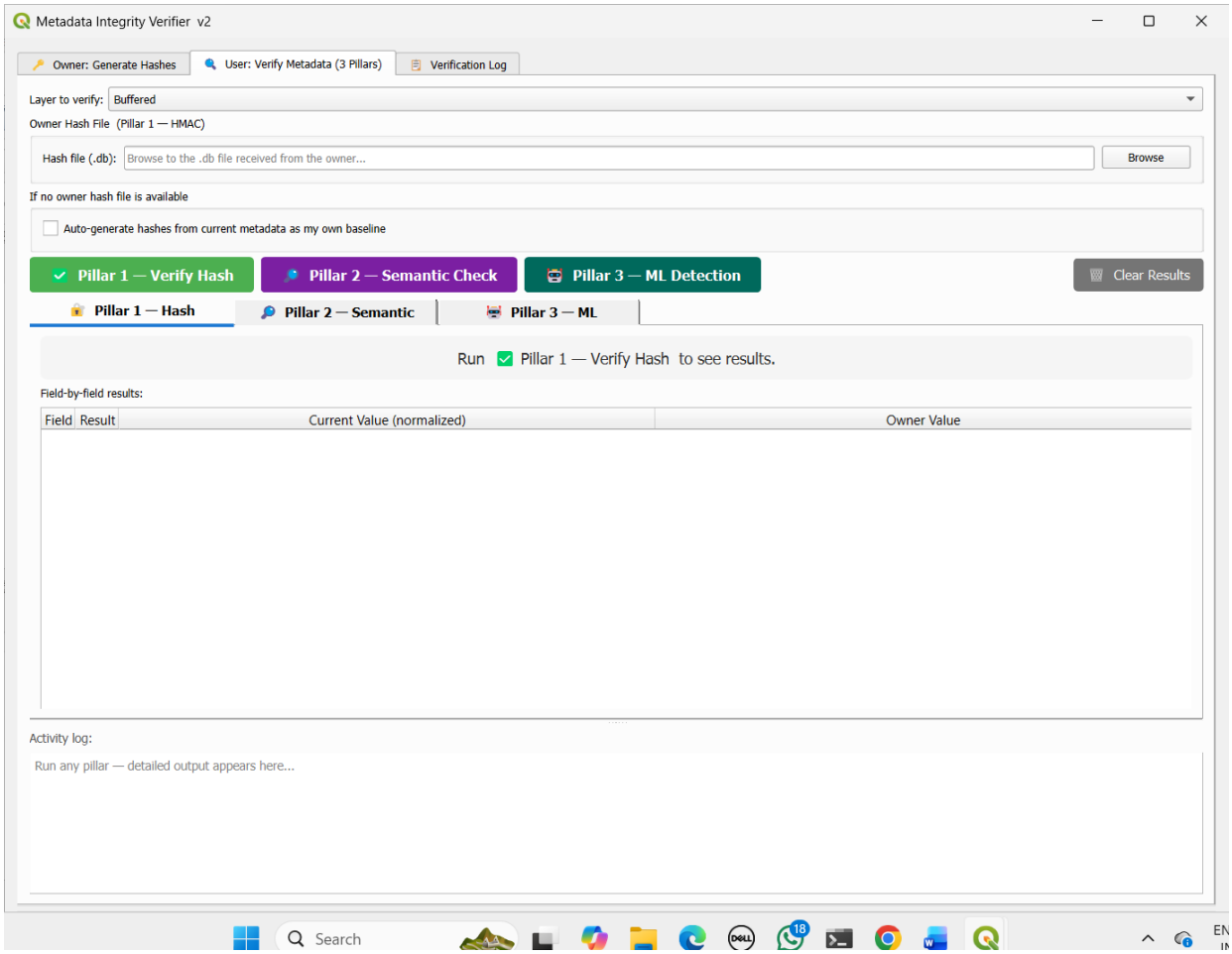


Figure 1: Metadata Verifier QGIS Plugin interface

Literature Review

Bertino and Thuraisingham [5] proposed one of the earliest comprehensive frameworks for GIS security, focusing primarily on access control mechanisms and the enforcement of security and privacy policies across multi-organizational geospatial data environments. While they acknowledged data integrity as a significant concern, their framework stopped short of offering a practical implementation for metadata verification, leaving a notable gap in ensuring the authenticity and traceability of geospatial datasets. This limitation points to the need for more operationally grounded approaches to GIS integrity assurance.

SHA-256, standardized under NIST FIPS 180-4 and formally specified in RFC 6234, is a well-established cryptographic hashing mechanism widely adopted for integrity verification across domains such as software package authentication, blockchain transactions, and file verification [2][3]. Its collision resistance and one-way properties make it particularly reliable for detecting unauthorized modifications to data. However, despite this proven utility, SHA-256 based verification has not been systematically applied to individual GIS metadata fields within plugin-based tooling environments, representing a practical gap that this work aims to address.

Methodology

The study was carried out in three phases. First, GIS analysis was performed on a vector layer to demonstrate the impact of metadata tampering on spatial results. In the second phase, a solution was proposed through the development of a QGIS plugin, Metadata Verifier, which operates in two modes: Owner Mode, for generating hash values of metadata fields, and User Mode, for verifying potential tampering by using three different model including hash verification if the original hash file is available, semantic checker and ML-detector. In the final phase, the performance of the hash-model and semantic model was evaluated using an automated Python script on a dataset of 21 layers, and the results were systematically recorded for analysis.

Figure2: Project flow chart

1. GIS Analysis –

The GIS analysis was carried out using QGIS, which was selected due to its open-source nature, flexibility, and extensive support for spatial data processing.

For the analysis, vector layer was generated using data obtained from OpenStreetMap (OSM). A vector layer of the area covering IITK was created.

All the metadata fields were field in accordance with the ISO 19115 standards. However, the QGIS has structured format for the metadata, and the metadata has to be filled in accordance with it. In QGIS, metadata is stored in .qmd file format, instead of standard .xml format.

After the successful creation of Building layer of the IITK, we have created a buffering layer of 10m.

Then the CRS in the metadata was changed from EPSG:32644 to EPSG:4326 manually, and the layer was open in a new project and tampered metadata file was uploaded.

Then all the settings were done on the basis of metadata, and the same GIS analysis was performed on this layer, a buffering layer of 10m was created.

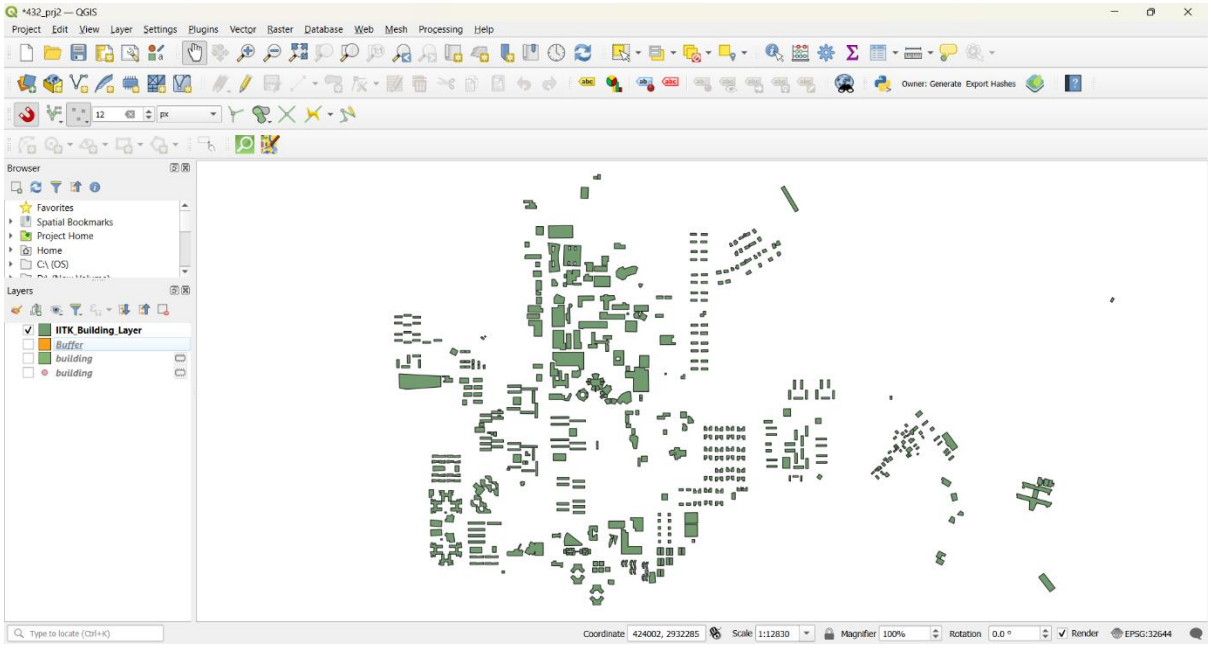


Figure 3: IITK Building layer created through OSM

2. Metadata Verifier Plugin –

The plugin operates in two modes. In Owner Mode, the data producer reads metadata from their QGIS layer, computes integrity fingerprints, and exports them into a portable hash database to be distributed with the layer. In User Mode, the recipient of the layer uses one or more of three independent verification pillars to check whether the metadata has been altered since the owner generated the fingerprints.

Pillar 1: Cryptographic Hash-Based Verification

In this, a python program has been written for the development of required model, as python code can easily be integrated in QGIS plugin. In storage mechanism, for secure storage and management of generated hashes and related metadata, the system utilizes SQLite as the underlying database management system. SQLite is chosen due to its lightweight, serverless architecture and ease of integration within the plugin environment.

- SHA-256 (Secure Hash Algorithm 256-bit) –

  SHA-256 is a one-way cryptographic hash function that maps an input of arbitrary length to a fixed 256-bit output called a digest. Its defining property for this application is collision resistance: it is computationally infeasible to find two different inputs that produce the same digest, and it is infeasible to reconstruct the input from the digest alone. In this system, SHA-256 serves as the underlying digest algorithm inside the HMAC construction. Every metadata field value, after normalization, is passed through the HMAC-SHA256 function to produce its fingerprint. If even a single character in a field value changes, the resulting digest changes completely and unpredictably, making any alteration detectable.

- HMAC (Hash-based Message Authentication Code) –

  A plain SHA-256 hash of a field value would be vulnerable to a substitution attack: an adversary who knows which fields are being hashed could precompute a valid hash for a replacement value and swap it into the database. HMAC prevents this by incorporating a secret key into the computation. The HMAC function combines the key and the input data in a specific two-pass construction and produces a digest that cannot be reproduced without knowing the key. In this system, each metadata field value is authenticated with HMAC-SHA256 using a key that is derived from the owner's password and is never stored in the hash database. A user who does not know the password cannot forge a valid hash for any field value, even if they have full access to the .db file.

- PBKDF2 (Password-Based Key Derivation Function 2) –

  Storing a raw password or a password hash in the database would be insecure. PBKDF2 solves this by transforming a human-chosen password into a cryptographically strong key through a process of iterated hashing. The function takes the password, a randomly generated salt, and an iteration count as inputs and applies an HMAC-SHA256 pseudorandom function repeatedly, in this system, 310,000 times to produce a 256-bit derived key. The high iteration count is intentional: it makes brute-force password guessing computationally expensive. The salt, which is unique per export and stored in the hash database, ensures that two exports with the same password produce different derived keys, defeating precomputation attacks such as rainbow tables. Critically, only the salt and iteration count are stored; the derived key itself is computed in memory during export and verification and is immediately discarded afterward.

- Value Normalization –

  Before any field value is passed to the hash function, it is normalized. The purpose of normalization is to ensure that the hash produced at export time and the hash recomputed at verification time are always identical for the same underlying content, regardless of minor formatting differences introduced by QGIS across versions or sessions. Normalization converts all text to lowercase, collapses whitespace sequences to a single space, strips leading and trailing whitespace, and treats None, NULL, and empty strings uniformly as the empty string.

- Combined Hash Construction –

  In addition to per-field hashes, a single combined hash is computed by concatenating all field-level HMAC digests in sorted alphabetical field order and passing this concatenated string through one additional HMAC computation. This combined hash serves as a single integrity fingerprint for the entire metadata record. During verification, the combined hash is checked first; only on a mismatch are individual field hashes compared one by one to identify exactly which fields were changed.

Steps involved in the model flow –

Owner Mode (Hash Generation)

- Generate a random salt and use it with the password to derive a secure key using PBKDF2.
- Generate a separate salt for the GIS layer.
- Select, sort, and normalize the metadata fields to ensure consistent formatting.
- Compute a secure hash for each field using HMAC with the derived key and layer salt.
- Combine all field hashes in a fixed order and generate a final combined hash.
- Store the salts, individual field hashes, and combined hash in the database, then discard the key from memory.

User mode (Verification)

- Use the stored salt and parameters to derive the same key from the user's password.
- Read and normalize current metadata field values.
- Recompute hashes for each field using the same method.
- Combine these hashes and generate a new final hash.
- Compare the new hash with the stored hash to verify integrity.

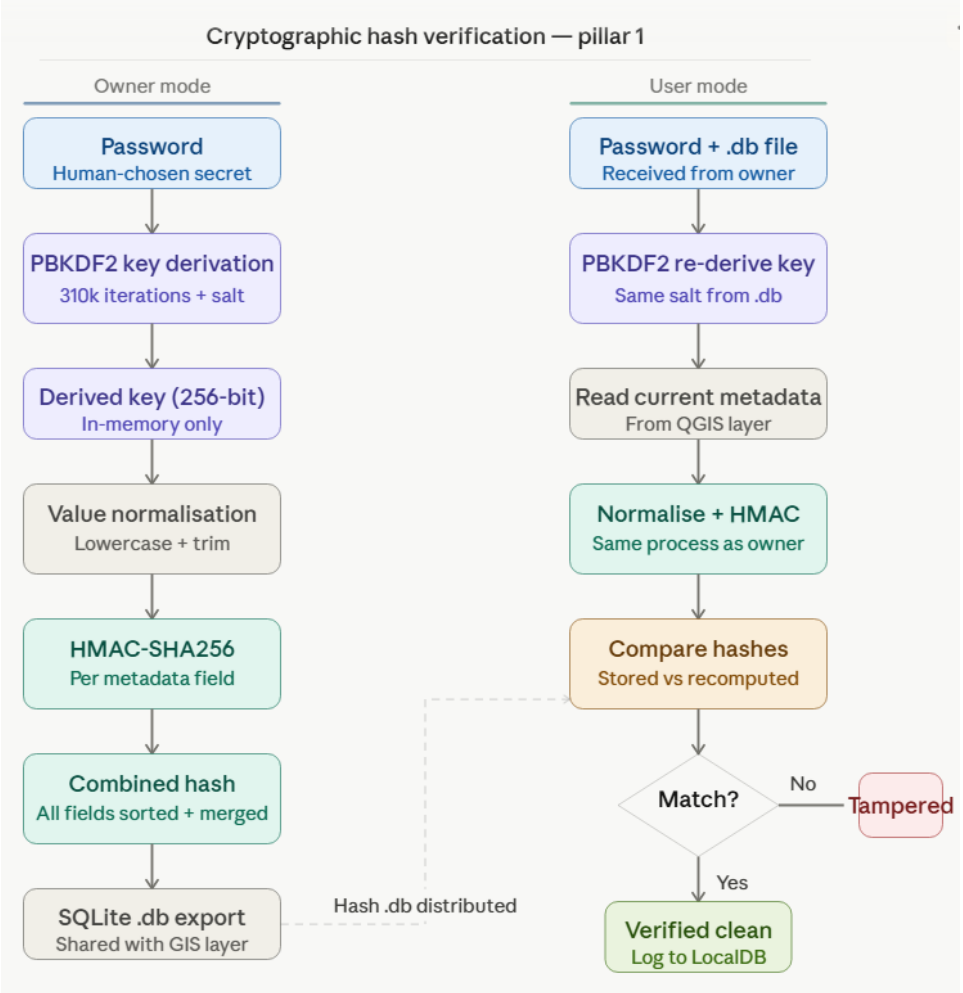


Figure 4: Hash-based verification model flow chart (Source: AI Generated)

Pillar 2: NLP-Based Semantic Consistency Checker

The semantic checker takes a fundamentally different approach. Rather than comparing against a stored reference, it asks whether the metadata record is internally coherent. An adversary who changes, say, the keywords or the abstract without updating the corresponding identifier or geometry type will produce a record where different fields contradict each other. The checker exploits these contradictions through a set of 14 weighted rules.

- Text Tokenization and Stopword Removal –

  All textual metadata fields are preprocessed before any semantic comparison. Tokenization splits text into individual word tokens using a regular expression that extracts only alphabetic sequences, discarding punctuation and digits. A domain-specific stopword list is then applied to remove words that carry no discriminative meaning in a GIS metadata context, common English function words as well as universally present GIS terms such as "data", "dataset", "layer", "created", and "modified". This ensures that subsequent comparisons are based on content-bearing vocabulary only.

- TF-IDF Cosine Similarity –

  Several rules require measuring the degree of topical overlap between two text fields, most notably Rule R10, which compares the abstract against the keywords. Raw word overlap (Jaccard similarity) was found to be insufficient because it treats all shared words equally regardless of their specificity. TF-IDF (Term Frequency–Inverse Document Frequency) weighting addresses this by assigning higher weight to words that appear frequently in one document but rarely across the corpus. Since the checker operates on only two documents at a time (abstract and keyword text), a two-document IDF is used. After weighting, the cosine similarity between the two TF-IDF vectors is computed. This metric ranges from 0 (completely disjoint vocabulary) to 1 (identical vocabulary distribution), and a very low cosine similarity on a substantive abstract is interpreted as evidence that the keywords have been replaced with topically unrelated terms.

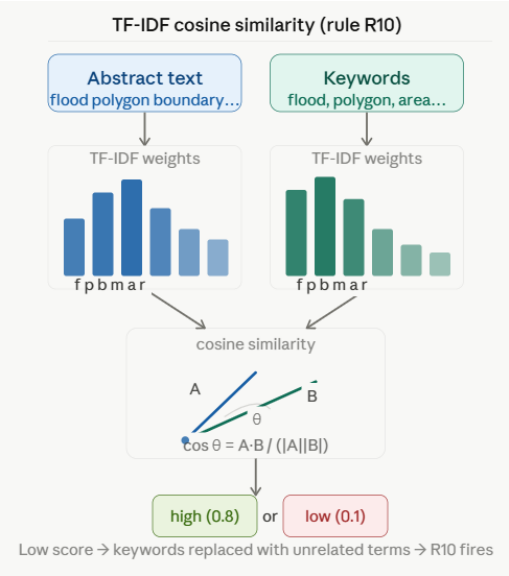


Figure 5: TF-IDF cosine similarity working mechanism (Source: AI Generated)

- Shannon Entropy for Vocabulary Assessment –

  Shannon entropy, borrowed from information theory, is used in two rules (R13 and also within the ML feature set) to assess the diversity of vocabulary in the abstract text. The entropy is computed over the probability distribution of word frequencies: a text with many repeated words has low entropy, while a text with diverse, varied vocabulary has high entropy. Low entropy in a metadata abstract suggests boilerplate, auto-generated, or copy-pasted content, which is a signal of low-quality or potentially fabricated metadata.

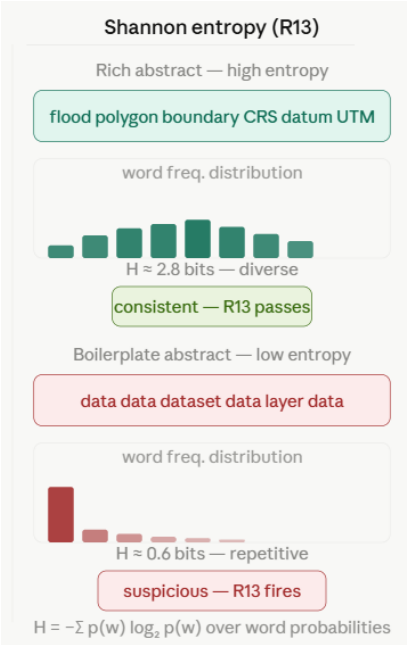


Figure 6: Shannon entropy working mechanism (Source: AI generated)

- Domain Vocabulary Matching –

  Rules R1–R3 rely on predefined domain vocabulary dictionaries that link geometry types with expected terms. For example, POINT is associated with terms like location or station, POLYGON with area or boundary, and LINESTRING with road or river. If a layer's declared geometry does not match the vocabulary used in its keywords, it indicates possible inconsistency or tampering. Additionally, a contradiction dictionary detects cases where the abstract contains terms incompatible with the declared geometry, such as describing a POLYGON layer as a linear network.

- Rule Weighting and Anomaly Score –

  Each of the 14 rules is assigned a weight reflecting its relative diagnostic importance. Rules dealing with CRS plausibility and date logic are assigned higher weights because these fields are unambiguous and their violations indicate clear tampering. Rules dealing with stylistic features such as keyword redundancy or template detection receive lower weights. When a rule fails, it contributes confidence × weight to the total anomaly score,

where confidence is a value between 0 and 1 returned by the rule to indicate how certain the violation is. The total score is capped at 1.0 and mapped to three verdict tiers: CONSISTENT (score ≤ 0.10), WARN (score ≤ 0.28), and SUSPICIOUS (score > 0.28).

The 14 Rules are:

Rule ID	Rule Name	What It Checks
R1	Geometry Keywords Check	Verifies if keywords contain hints about the layer's geometry type
R2	Identifier–Geometry Consistency	Ensures identifier terms match the actual geometry type
R3	Abstract Contradiction Check	Detects contradictions in the abstract related to geometry
R4	Identifier–Keyword Overlap	Checks if important identifier terms appear in keywords
R5	Date Consistency	Ensures creation date is not later than publication date
R6	CRS vs Dataset Scope	Verifies if CRS matches dataset scope (global vs projected)
R7	Suspicious Content Scan	Detects unusual or malicious patterns in text
R8	Contact Consistency	Checks if all contact emails belong to a single domain
R9	Keyword Quality Check	Evaluates keyword diversity and avoids excessive keywords
R10	Abstract–Keyword Similarity	Measures semantic similarity between abstract and keywords
R11	Spatial Validity Check	Ensures bounding box and CRS values are geographically valid
R12	Feature Count Plausibility	Validates if feature count is realistic for dataset type
R13	Boilerplate / Redundancy Check	Detects repetitive or low-information metadata
R14	Domain–Field Consistency	Ensures claimed domain matches actual field attributes

Figure 7: Rules for checking consistency in metadata

Steps involved in the model flow –

- Extract all relevant metadata fields from the layer into a structured record.
- Initialize an anomaly score to measure inconsistency.
- Evaluate a set of predefined rules on the metadata to check for logical and semantic consistency.
- For each rule, determine whether it passes or fails, along with a confidence level and explanation.
- If a rule fails, add its weighted contribution to the anomaly score; otherwise, add nothing.
- Accumulate contributions from all rules to compute the overall anomaly score.
- Limit the final anomaly score to a maximum value of 1.
- Compare the anomaly score against predefined thresholds:
    - Low score indicates consistent metadata
    - Medium score indicates a warning
    - High score indicates suspicious or inconsistent metadata
- Return the final verdict, overall anomaly score, and detailed results for each rule to explain which checks passed or failed.

Pillar 3: ML-Based Anomaly Detection

The third pillar uses trained machine learning models to score a metadata record against statistical patterns learned from real GIS metadata. Unlike the previous two pillars, it makes no comparison to a stored baseline and applies no handcrafted rules. It instead asks: does this metadata record look like it belongs to the population of normal GIS metadata records?

- Isolation Forest –

  Isolation Forest is an unsupervised anomaly detection algorithm that exploits the observation that anomalous records are easier to isolate than normal ones. The algorithm builds an ensemble of random decision trees (isolation trees), each constructed by recursively partitioning the feature space using randomly chosen features and random split thresholds. For any given record, the algorithm measures the average number of partitions required to isolate that record across all trees. Normal records, which cluster together in feature space, require many partitions to isolate; anomalous records, which occupy sparse regions of feature space, are isolated quickly. The isolation score is inverted and normalized so that a higher output value indicates a more anomalous record.

- One-Class SVM –

  The One-Class Support Vector Machine learns a boundary in a high-dimensional feature space that encloses the region occupied by normal training records. Records that fall outside this boundary at inference time are classified as anomalous. The kernel function (in this case RBF, or Radial Basis Function) implicitly maps the input features to a much higher-dimensional space where this boundary is a hyperplane. The One-Class SVM's boundary adapts to the shape of the normal data distribution in feature space rather than assuming any fixed parametric distribution.

- Ensemble and Verdict –

  The two models are combined in a weighted ensemble. The final tamper probability is computed as 0.35 × (Isolation Forest probability) + 0.65 × (One-Class SVM probability). The higher weight on the One-Class SVM reflects its stronger performance during internal validation. The ensemble tamper probability is mapped to three verdicts: CLEAN (below 35%), UNCERTAIN (35–60%), and SUSPICIOUS (above 60%).

- Feature Engineering –

  The ML model operates on 19 numeric features derived from metadata fields: abstract length, abstract word count, abstract vocabulary entropy, title length, title word count, TF-IDF cosine overlap between title and abstract, bounding box coordinates (west, east, south, north), bounding box width, height, area, centre latitude and longitude, creation date year and month, CRS category encoded as an ordinal integer, and keyword count. All features are standardized using a Standard Scaler fitted on the training data before being passed to the models.

Steps involved in the model flow –

- Extract metadata from the GIS layer, similar to the semantic checker.
- Compute a set of numerical features from the metadata and arrange them in the same order used during training.
- Scale the feature vector using a pre-fitted standardization method to ensure consistency with the trained models.
- Pass the scaled data through two anomaly detection models:
  - Isolation Forest to detect outliers based on data isolation
  - One-Class SVM to learn the boundary of normal data
- For each model:
  - Compute an anomaly score
  - Determine whether the data is flagged as abnormal
  - Normalize the score into a probability-like value
- Combine the outputs of both models using weighted averaging to get a final tampering probability.
- Compare this probability against predefined thresholds to classify the result as:
  - Clean (low probability)
  - Uncertain (moderate probability)
  - Suspicious (high probability)
- Return the final verdict along with the computed tampering probability.

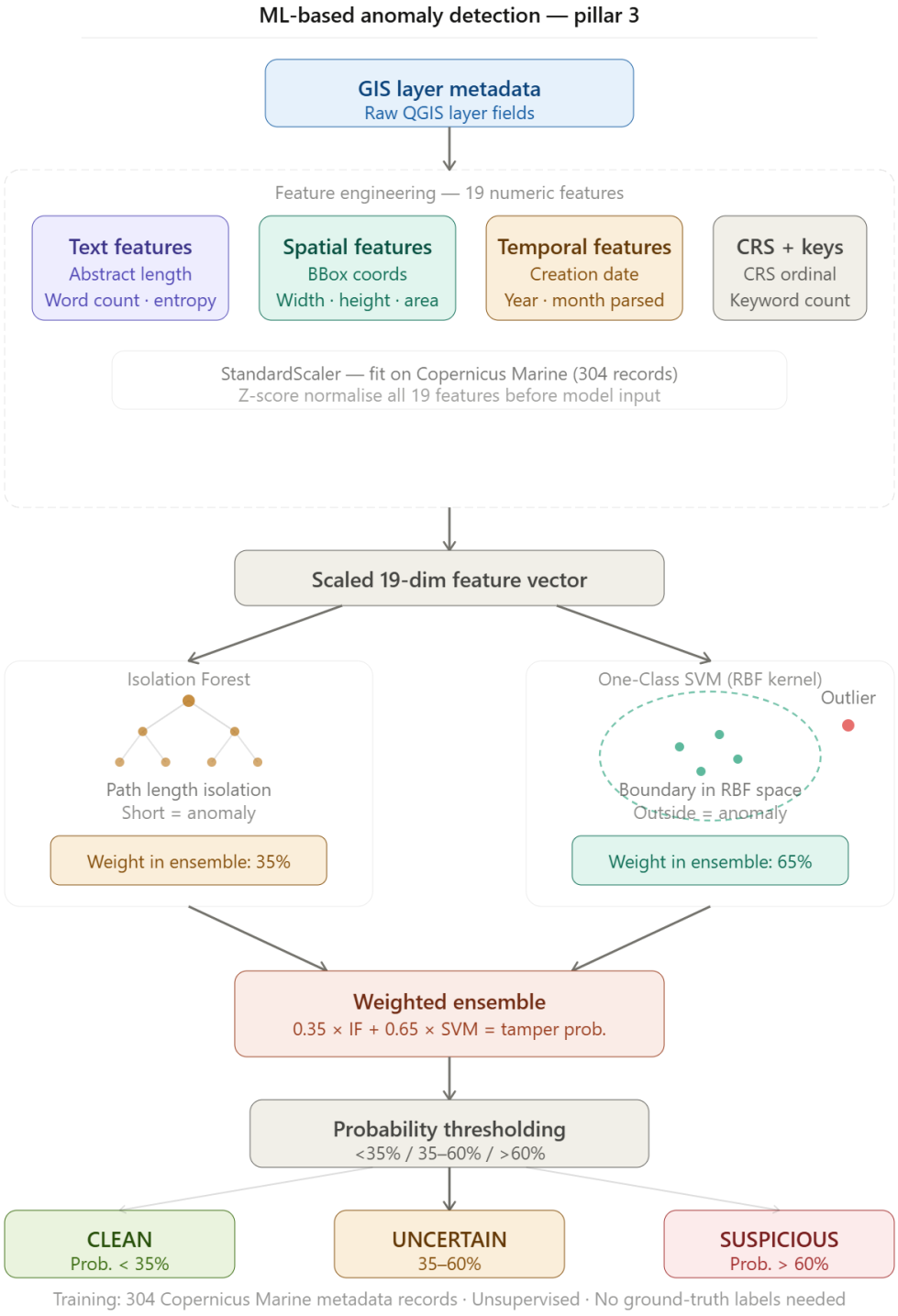


Figure 8: ML-detector verification model flow chart (Source: AI generated)

3. Automated Evaluation –

A dataset of 21 layers was prepared for the evaluation of the hash-based verification and semantic-checker.

The layers were downloaded from the open source, the official website of Natural Earth Data. The 21 layers were downloaded and then metadata was manually filled and tampered. Twelve layers were clean and nine were tampered under various single-field and multi-field scenarios.

The script operated in three phases: registering baseline hashes from clean reference files (not required for semantic checker), verifying all 21 test layers blindly against those baselines without any ground truth knowledge, and then computing performance metrics using ground truth only for scoring. Metrics recorded included accuracy, sensitivity, specificity, precision, F1 score, and per-layer processing time.

For the evaluation of ML detector, dataset was downloaded from Copernicus Marine CSW, with 306 metadata entries. And the same metric evaluation was done for this ML model also.

Different metric used to check model performance are:

Accuracy: [(TP + TN) / total] Proportion of total predictions that are correct. It measures overall correctness but can be misleading if the dataset is imbalanced.

Recall (Sensitivity): [TP / (TP + FN)] Proportion of actual tampered cases that are correctly identified. It measures how well the system detects true tampering.

Specificity: [TN / (TN + FP)] Proportion of actual clean cases that are correctly identified. It measures how well the system avoids false alarms.

Precision: [TP / (TP + FP)] Proportion of predicted tampered cases that are truly tampered. It reflects the reliability of positive alerts.

F1 Score: [2 × precision × recall / (precision + recall)] Harmonic mean of precision and recall. It provides a balanced measure when both false positives and false negatives matter.

Steps involved in the evaluation flow –

- Load dataset containing multiple GIS layers with metadata.
- For each layer, extract required metadata fields.
- Prepare Ground truth csv file, assign true labels to each sample (e.g., clean or tampered).
- For each layer in dataset:
    - Prepare input (normalize text, extract features if required).
    - Pass data through the model (hash / semantic / ML).
    - Store predicted result and any scores.
- For each sample:
    - Compare model output with ground truth label.
    - Record whether prediction is correct or incorrect.
- Calculate performance measures such as:
    - Accuracy (overall correctness)
    - Precision and recall (for tampering detection)
    - False positives and false negatives
- For ML detector:
    - Extract features and scale using trained parameters.
    - Run through trained models and compute anomaly score.

    - Convert score to probability and assign class label.
    - Evaluate predictions using same metrics as above.
- Summarize performance across all samples.
- Report final evaluation metrics for each model.

Implementation

Plugin File Structure

- The plugin is organized as a standard QGIS Python plugin package. The entry point is metadata_verifier.py, which registers the plugin with QGIS, adds menu items and a toolbar button, and instantiates the dialog.
- All GUI logic is contained in dialog.py.
- Cryptographic logic is separated into hash_engine.py. Metadata reading is in metadata_reader.py. Database management is in db_manager.py.
- The NLP semantic engine is in semantic_checker.py.
- The ML inference module is in ml_detector.py. The shared constants (field definitions, database paths) are in constants.py. The pre-trained model bundle is shipped as model_bundle.pkl.
- All long-running operations (export, verification, semantic check, ML detection) are run in QThread background worker classes to keep the QGIS UI responsive.

Hash Engine (hash_engine.py) –

- Salt Generation
    - Generate a secure 32-byte random salt using `os.urandom` and store it as a hex string.
    - Ensures uniqueness and protects against precomputed attacks.
- Key Derivation
    - Convert password to bytes and derive a 256-bit key using PBKDF2 with SHA-256 and high iterations.
    - Output is stored in hexadecimal format.
- Normalization
    - Clean metadata values by handling nulls, trimming spaces, converting to lowercase, and standardizing whitespace.
    - Ensures consistent input before hashing.
- HMAC Computation
    - Combine layer salt with normalized text and compute a secure hash using HMAC (SHA-256 based).
    - Produces a deterministic hex digest for integrity verification.
- Hashing All Fields
    - Process fields in sorted order for consistency.
    - Generate individual HMAC hashes for each field.
    - Concatenate all hashes and compute a final combined hash.
    - Return field hashes, combined hash, and normalized values.

Metadata Reader (metadata_reader.py) –

- CRS Extraction: Retrieve the coordinate reference system as a stable EPSG identifier (e.g., code format), avoiding descriptive text to ensure consistency.
- Geometry Type: Extract the geometry type of the layer and convert it into a readable string format.
- Bounding Box
    - Obtain the spatial extent of the layer and format it as xmin, ymin, xmax, ymax.
    - Round each coordinate to a fixed precision (6 decimal places) for uniformity.
- Contact Information
    - Combine contact name and organization into a single string format.
    - Sort all entries and join them into a single standardized string.
- Keywords
    - Merge all keyword groups into one list.
    - Clean, remove empty values, sort, and combine into a single string.
- Lineage: Combine metadata history entries into one string while preserving their original order, as sequence is meaningful.
- Accuracy / Constraints
    - Extract relevant constraint values (excluding scale and resolution).
    - Format each as type and value, then sort and combine into a single string.
- Creation Date
    - Search metadata dates to find the creation date entry.
    - Return the date if available, otherwise return an empty value.

Database Manager (db_manager.py) –

Two separate SQLite databases are maintained.

HashDB (the owner's shareable export file):

- PBKDF2 Salt Initialization
    - Generate a new salt and store it in the database.
    - This salt is later used for deriving secure keys.
- Key Derivation
    - Retrieve stored salt and iteration count.
    - Derive a secure key from the user password using PBKDF2.
- Store Layer Hash
    - Save or update the combined hash, layer salt, and field list for a given layer.
    - Ensures one consistent record per layer.
- Store Field Hash
    - Save or update individual field hash values along with their normalized data.
    - Enables fine-grained tampering detection.
- Retrieve Layer Hash
    - Fetch stored layer-level hash data for verification.
    - Returns no result if the layer is not found.
- Retrieve Field Hashes
    - Fetch all stored field hashes for a given layer.
    - Returned in a structured format for comparison during verification.

LocalDB (the user's private log, never shared):

- tamper_log: Stores verification results including layer name, timestamp, verdict, computed and stored hashes, list of tampered fields, and any additional notes. Used for auditing and tracking verification history.
- When a verification is performed, store the result in the tamper log.
- Include the current timestamp, comparison results, hashes, and any detected tampered fields.

Semantic Checker (semantic_checker.py) –

- Metadata Extraction
  - Extract all relevant information from the layer into a unified record.
  - This includes geometry type, CRS identifier, spatial extent, feature count, field names, keywords, contacts, emails, organizations, and important dates.
  - Normalize values (e.g., lowercase text, base geometry type) to ensure consistency.
- Text Processing
  - Tokenize textual fields by extracting meaningful words using pattern matching.
  - Remove common stopwords and very short tokens to focus on relevant terms.
- Similarity Computation
  - Convert textual data (such as abstract and keywords) into numerical representations using TF-IDF.
  - Compute similarity between texts using cosine similarity to assess semantic alignment.
- Rule Evaluation
  - Apply a predefined set of semantic validation rules to the extracted metadata.
  - Each rule returns whether it passed, along with a confidence score and explanation.
  - If a rule fails, its weighted contribution is added to the anomaly score.
- Anomaly Scoring
  - Accumulate contributions from all rules to compute a final anomaly score.
  - Cap the score at a maximum value of 1 to maintain a normalized scale.
- Compare the anomaly score against predefined thresholds to classify the metadata as:
  - Consistent
  - Warning
  - Suspicious
- Return the final verdict, overall anomaly score, and a detailed per-rule breakdown for transparency.

ML Detector (ml_detector.py) –

- Model Bundle
  - Contains two trained models: Isolation Forest and One-Class SVM.
  - Includes a fitted scaler for feature normalization, ordered feature list, and score ranges for normalization.
  - Also stores training metadata such as source and dataset size.
- Feature Engineering
  - Text-Based Features
    - Compute length, word count, and vocabulary entropy of abstract and title.

        - Measure overlap between title and abstract to capture semantic consistency.
    - Spatial Features: Extract bounding box coordinates and derive width, height, area, and center location.
    - Temporal Features: Parse creation date to extract year and month.
    - CRS Encoding: Convert CRS identifiers into numeric categories based on projection type.
    - Keyword Features: Count total keywords as a measure of metadata richness.
- Preprocessing
    - Arrange features in the same order as training.
    - Scale them using the pre-fitted standardization model to ensure compatibility with trained models.
- Model Evaluation
    - Pass scaled features into both models to obtain anomaly scores and predictions.
    - Normalize raw scores using stored min–max ranges and invert them so higher values indicate higher anomaly likelihood.
- Ensemble Decision
    - Combine both model outputs using weighted averaging to compute a final tampering probability.
    - Assign verdict based on thresholds:
        - Low probability → Clean
        - Medium → Uncertain
        - High → Suspicious
- Output: Return final verdict, tampering probability, individual model contributions, and extracted feature values for transparency.

Results

GIS Analysis –

Buffering analysis was done on the IITK Building layer, and it was observed that due to tampering in the CRS, the buffering result was irrelevant on tampered layer. The area calculation also showed nan value.

Although, the metadata itself does not affect GIS analysis, because in the software like QGIS and ArcGIS, for the GIS analysis purposes, CRS information is taken from the layer itself, not from the metadata file, so tampering done alone in metadata does not impact GIS analysis directly. But it triggers users or mislead them to take wrong decision, like changing CRS of layer by trusting metadata information.

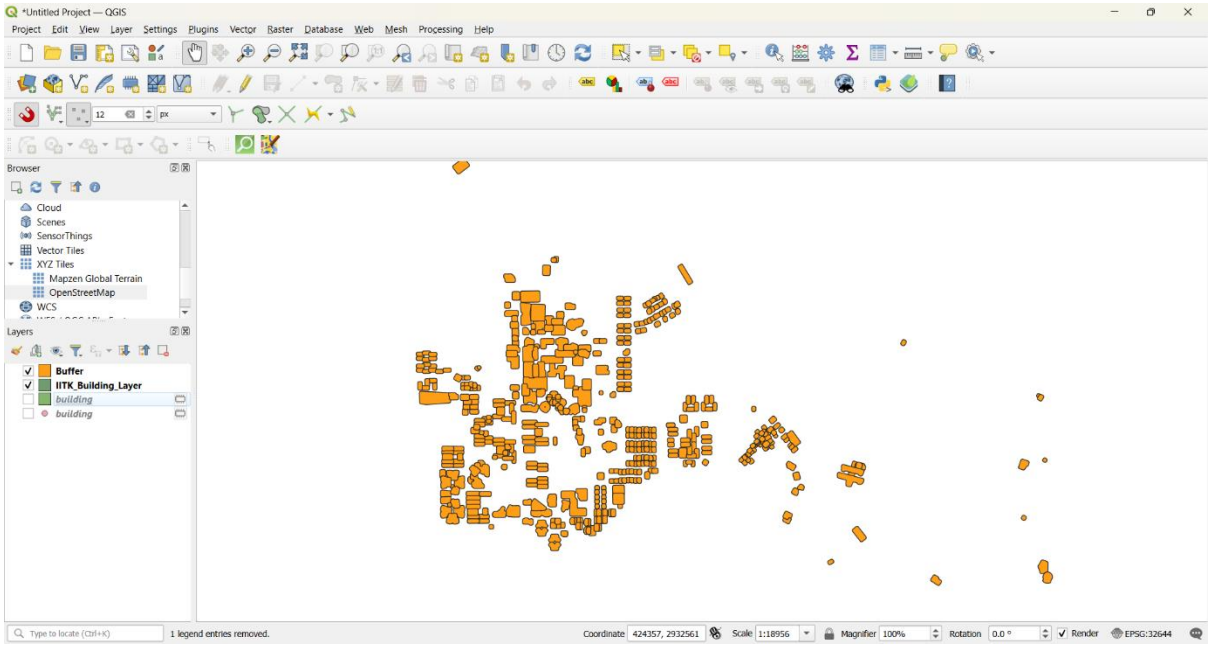
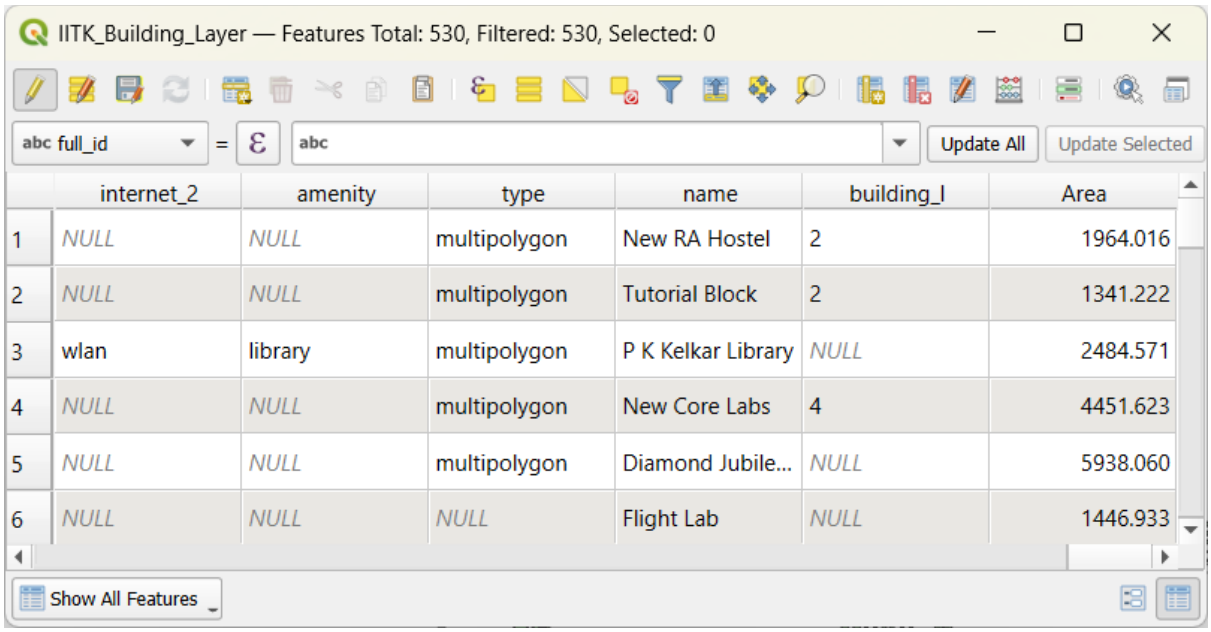


Figure 9: Buffering on clean IITK_Building_layer



	internet_2	amenity	type	name	building_l	Area
1	NULL	NULL	multipolygon	New RA Hostel	2	1964.016
2	NULL	NULL	multipolygon	Tutorial Block	2	1341.222
3	wlan	library	multipolygon	P K Kelkar Library	NULL	2484.571
4	NULL	NULL	multipolygon	New Core Labs	4	4451.623
5	NULL	NULL	multipolygon	Diamond Jubile...	NULL	5938.060
6	NULL	NULL	NULL	Flight Lab	NULL	1446.933

Figure 10: Area calculation of the clean layer

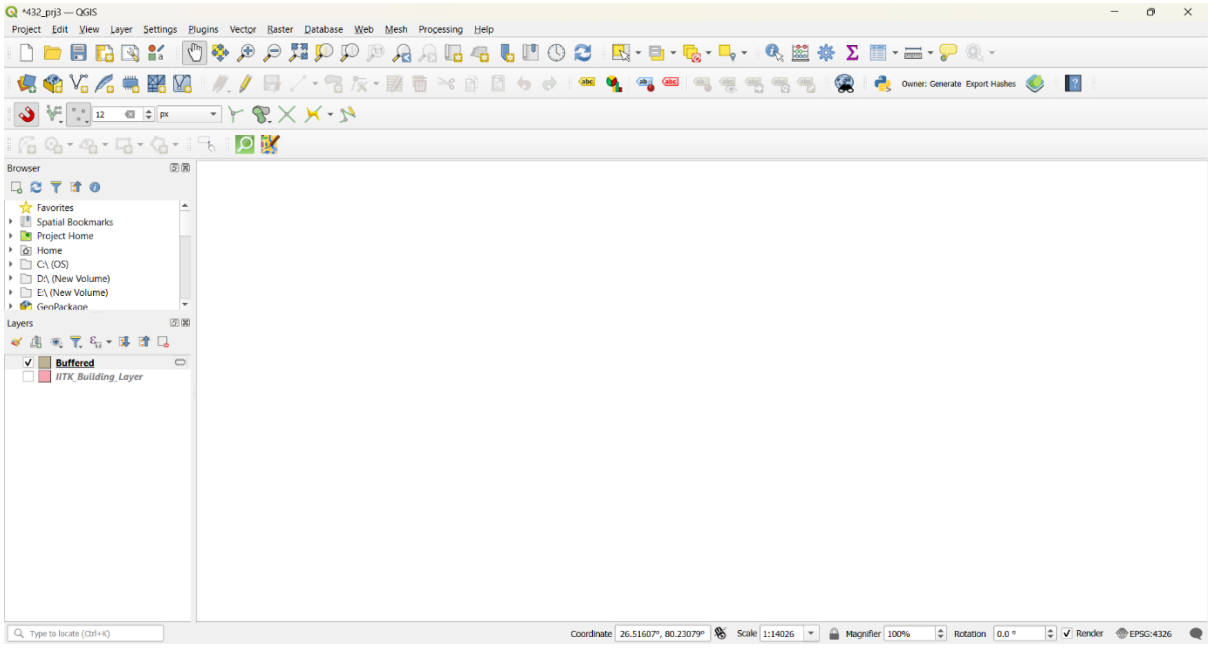


Figure 11: Buffering on the tampered IITK_Building_layer

IITK_Building_Layer — Features Total: 530, Filtered: 530, Selected: 0

abc full_id = ε abc Update All Update Selected

	internet_2	amenity	type	name	building_l	Area
1	NULL	NULL	multipolygon	New RA Hostel	2	nan
2	NULL	NULL	multipolygon	Tutorial Block	2	nan
3	wlan	library	multipolygon	P K Kelkar Library	NULL	nan
4	NULL	NULL	multipolygon	New Core Labs	4	nan
5	NULL	NULL	multipolygon	Diamond Jubile...	NULL	nan
6	NULL	NULL	NULL	Flight Lab	NULL	nan

Show All Features

Figure 12: Area calculation of the tampered layer

This is due to change in the units from meter in EPSG:32644 to degree in EPSG:4326. Due to this complete GIS analysis don't make any sense.

Hash-Based Verification –

This is a cryptographic based hash verification system. It is deterministic model, which compares hash of the metadata fields, and little change in that can be detected by it.

The evaluation shows that out of 9 tampered layers, 2 were not detected by it. These layers were tampered in Date and Email information. The model was not able to detect these because the metadata fields that were selected for the generation of hash does not include these fields so their hash was not generated, therefore model didn't detect them.

Other than this, the model worked perfectly, and the all performance matrix show good performance as per the expectation from the model. This limitation arises from the specific implementation (i.e., hard-coded selection of fields) rather than any flaw in the underlying methodology. Incorporating these fields into the hashing logic would likely enable correct detection of such tampering instances.

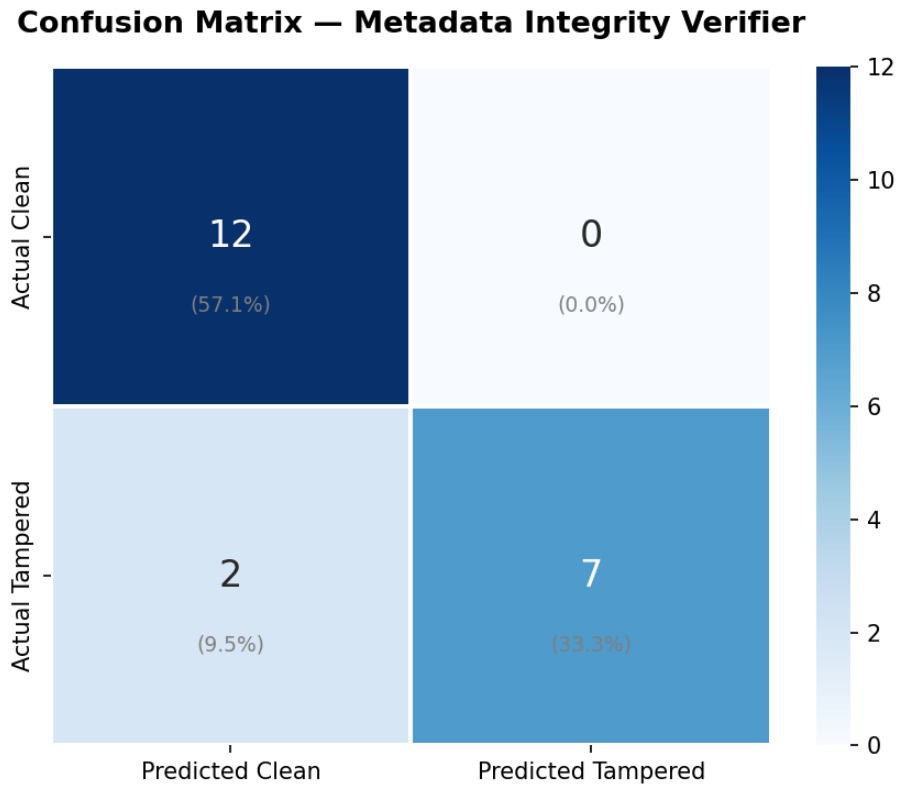


Figure 13: Confusion matrix for hash-based model

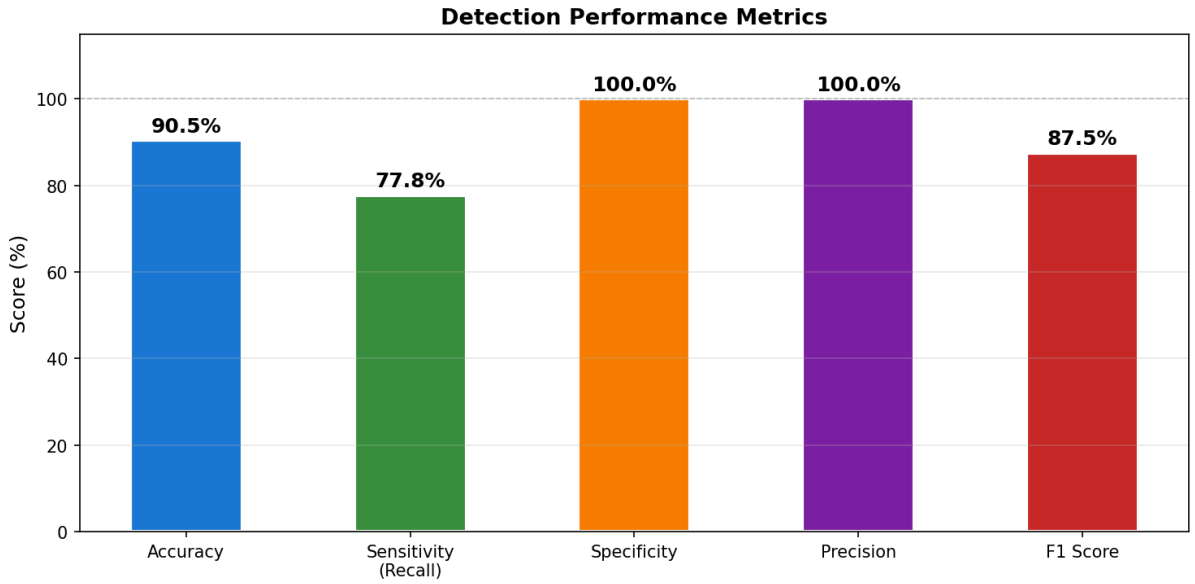


Figure 14: Detection performance metrics graph of hash-based model

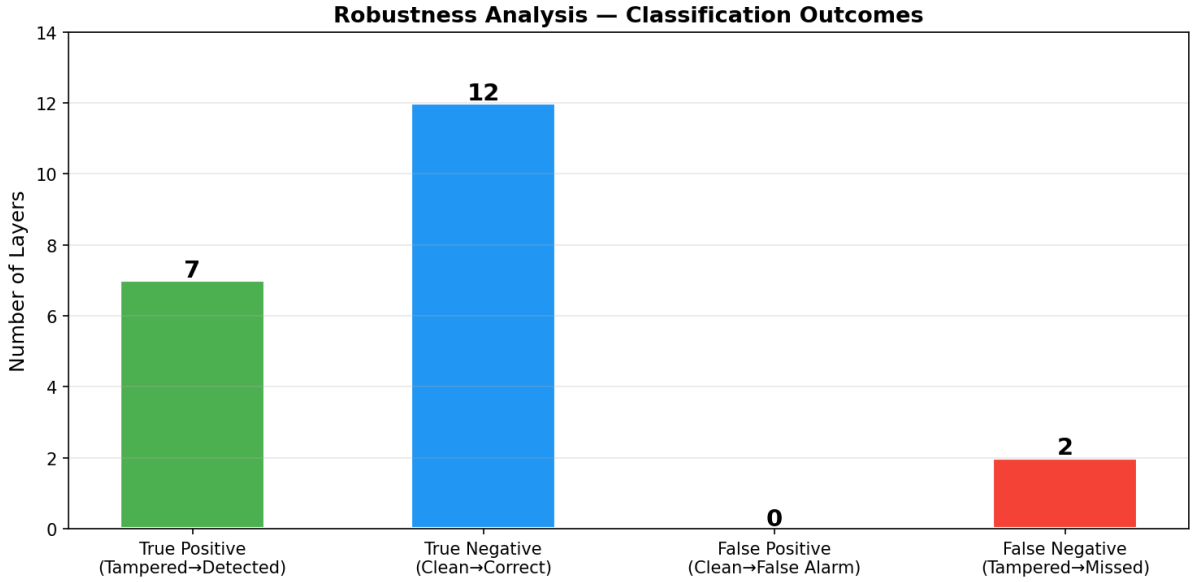


Figure 15: Robustness analysis of hash-based model

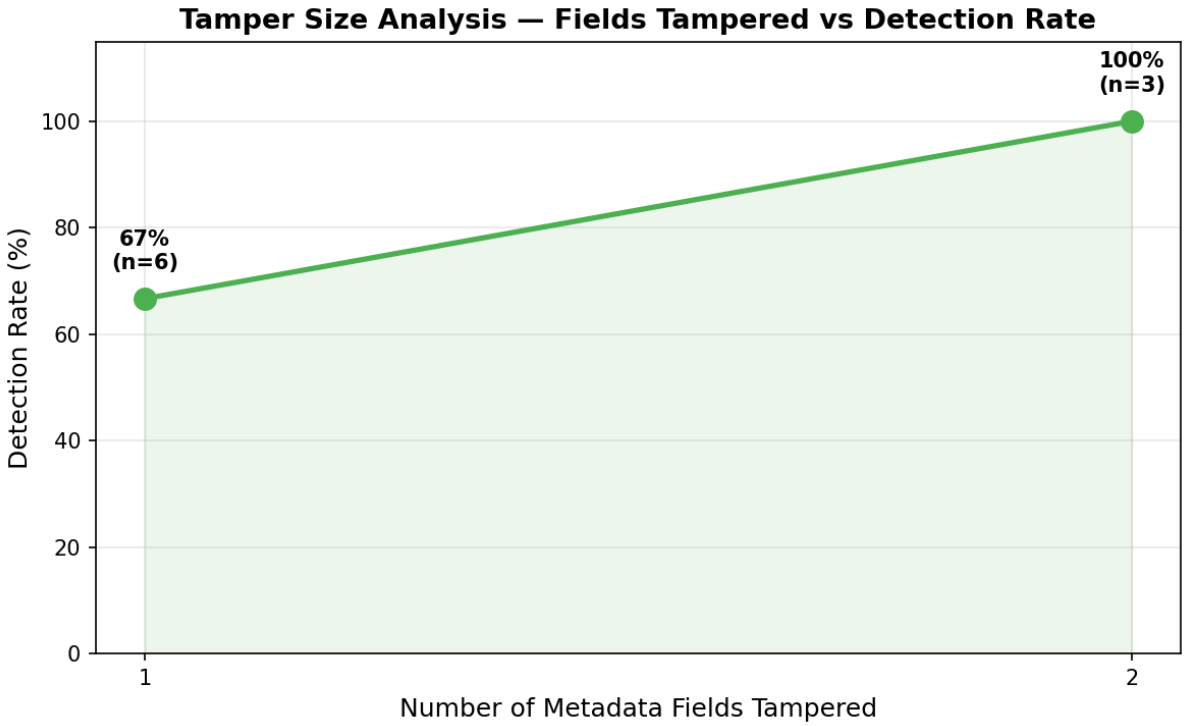


Figure 16: Tamper size analysis of hash-based model

Semantic Checker –

The accuracy of Semantic checker is 66.7%, while Sensitivity is 22.2%, which shows it does not wrongly displayed any clean layer as tampered (FP =0).

Precision and specificity of 100% shows it does not wrongly detect any tampered layer and all the clean layer were correctly identified as clean.

The important point is out of all the rules, only firing of R6 and R7 rule results into tampering detection, which shows different weightage assigned to all the rules and the pre described threshold level.

The evaluation performance show that the intentional tampering can even bypass the logical rules and remain undetected. These rules only catch any inconsistency and illogical sense from metadata, but evaluation shows metadata was either more of consistent or the code logic and implementation need to be improved.

	Plugin Says Clean	Plugin Says Tampered
Actually Clean	12	0
Actually Tampered	7	2

Figure 17: TP, TN, FP, FN table for semantic checker model

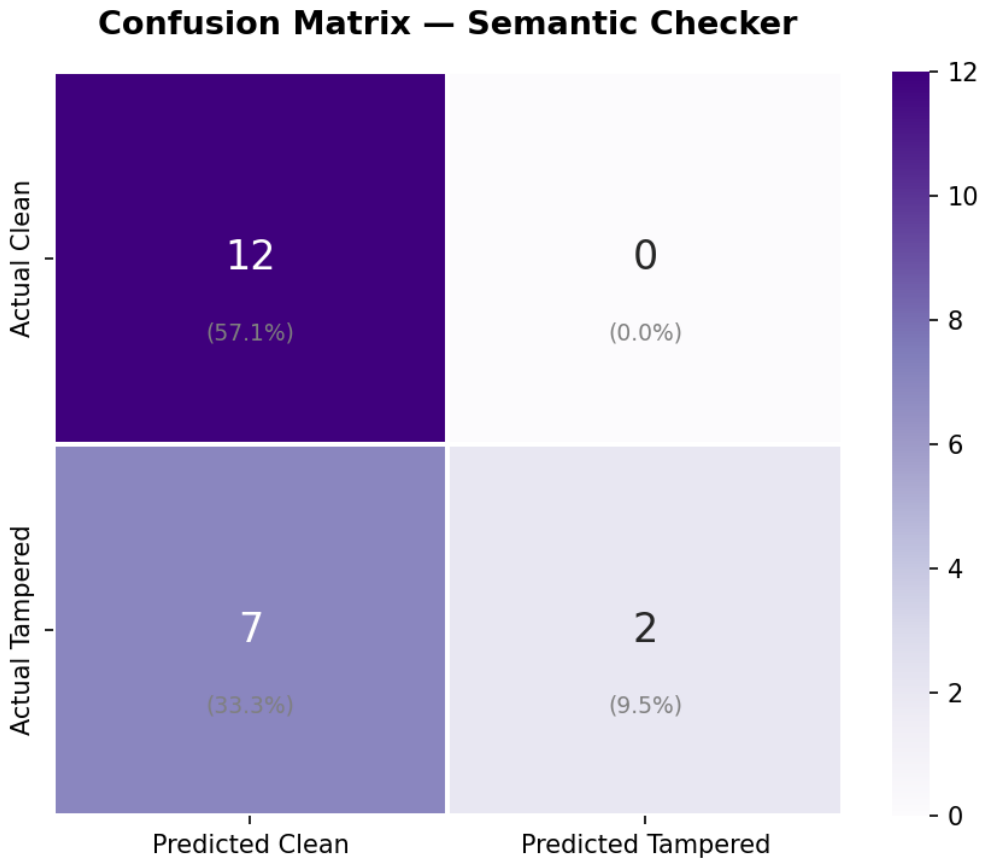


Figure 18: Confusion matrix of Semantic checker model

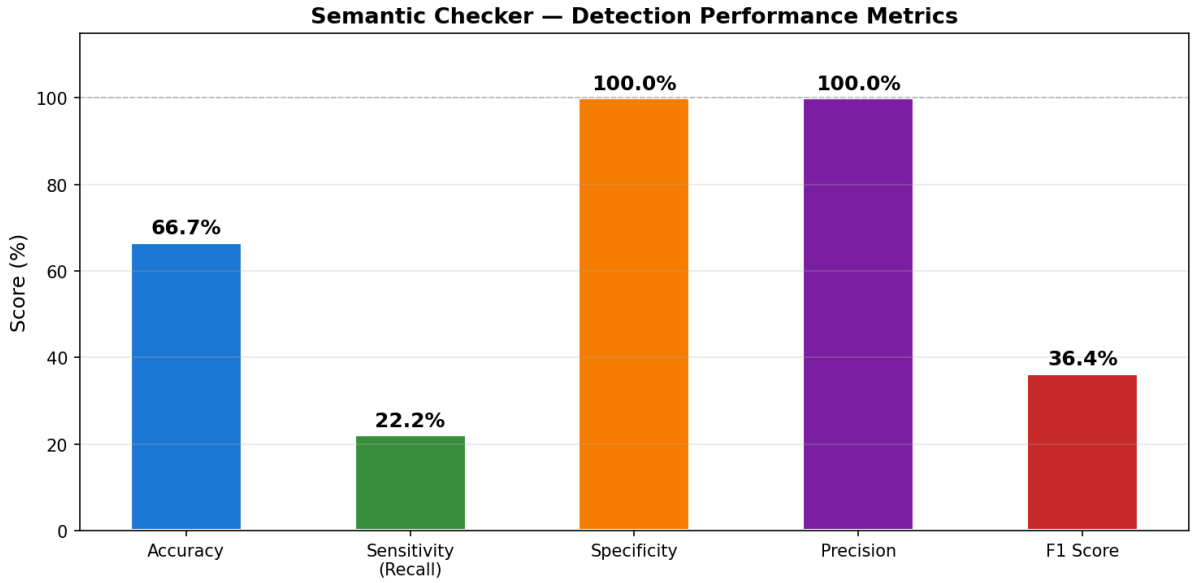


Figure 19: Detection performance metrics of semantic checker

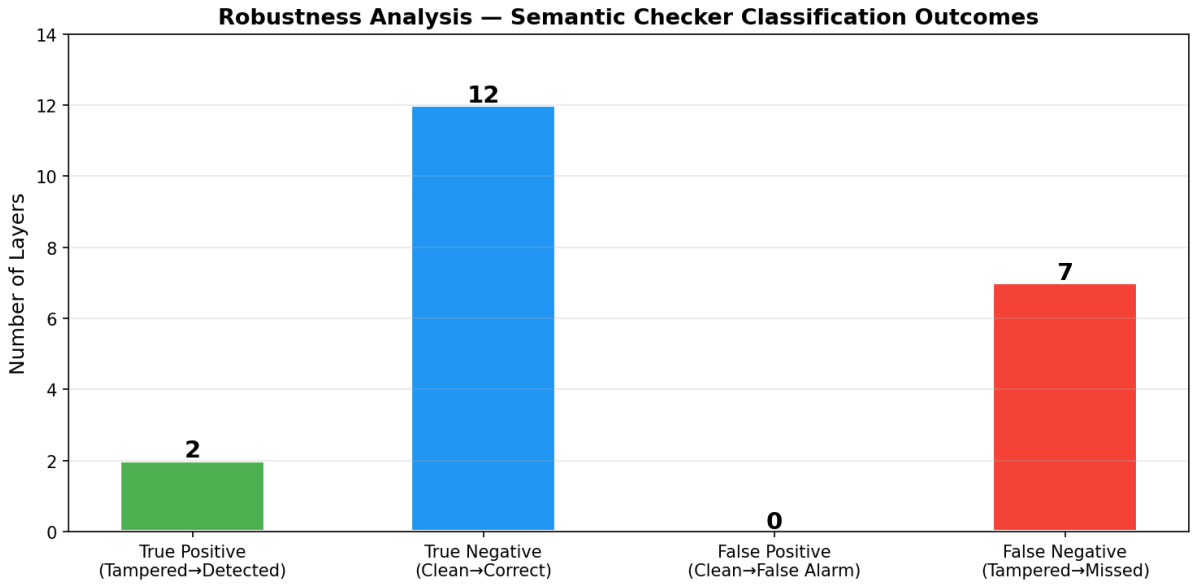


Figure 20: Robustness analysis of semantic checker

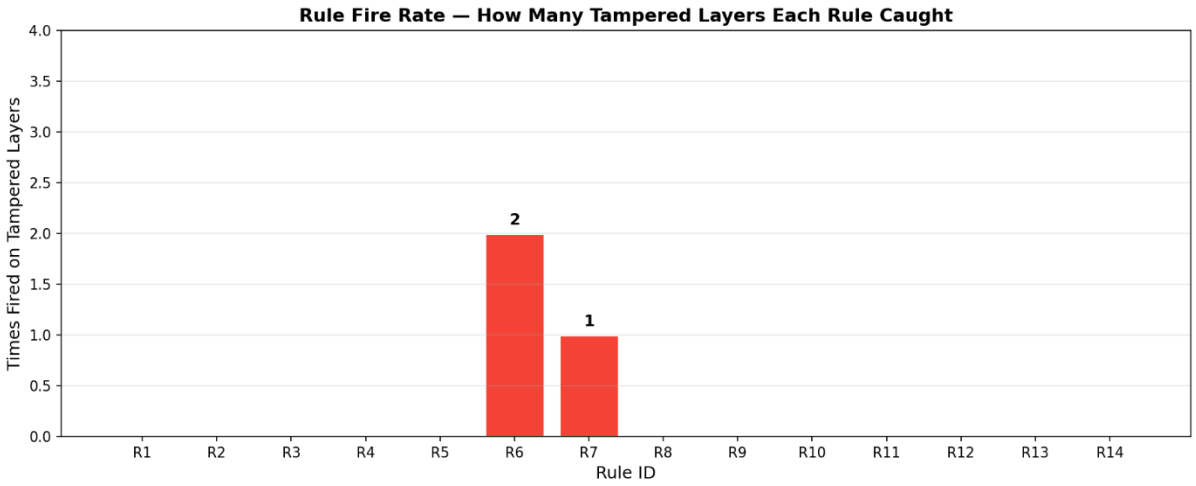


Figure 21: Rule fire rate detection in tampering graph

ML Detector –

Two rows had missing bbox values and were dropped, leaving 304 clean training records.

One anomaly in the raw data itself: Rows 180/181 already have out-of-range bbox values (bbox_east=1080, bbox_south=228) in the original file. These are genuinely unusual records that the model correctly flags. This is a sign the model is working, it catches real anomalies too, not just synthetic ones.

Clearly, from the evaluation it is visible that One-Class SVM is the stronger model, it caught 66 of 90 tampered records with only 13 false positives. The Isolation Forest struggled more, because of only 304 training samples, the SVM's tight boundary approach outperforms the tree-based partitioning.

Metric	Isolation Forest	One - Class SVM	Ensemble
Recall (tampered caught)	50%	73.30%	77.80%
Precision	61.60%	83.50%	69.30%
F1 Score	55.20%	78.10%	73.30%
False Positive Rate	13%	6%	14.40%

Figure 22: Model vs metrics comparison table for ML detector

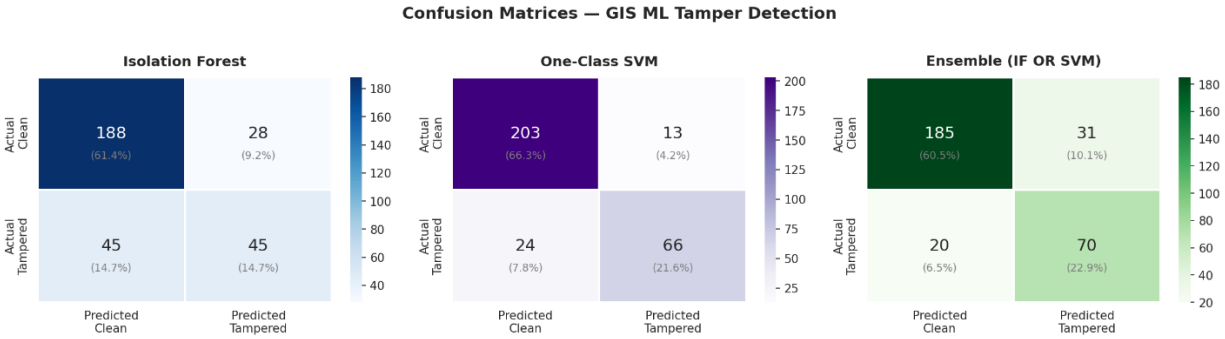


Figure 23: Confusion matrix for different model in ML detector

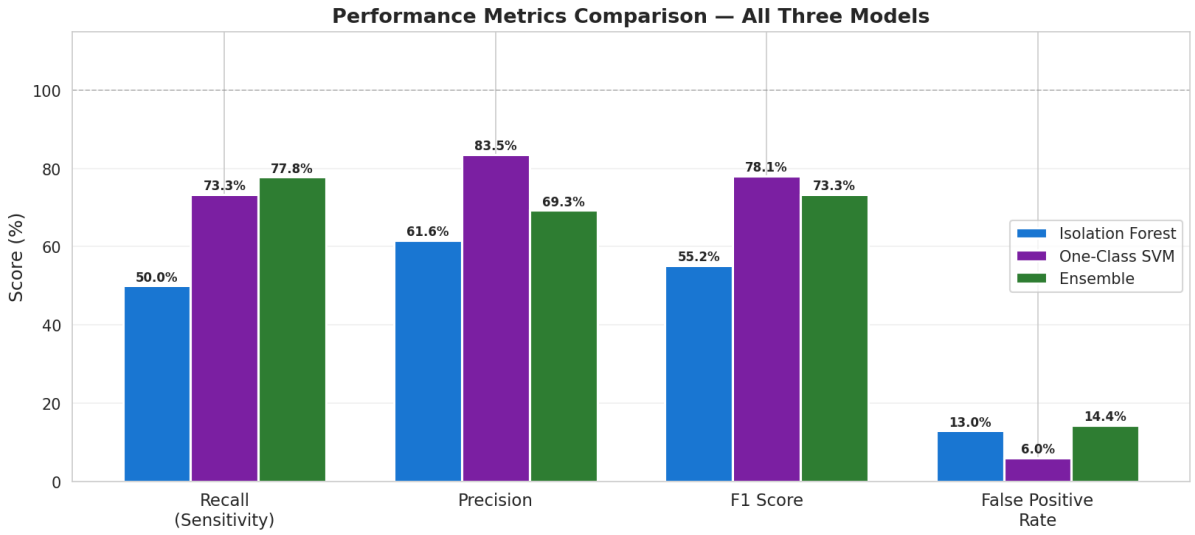


Figure 24: Performance analysis of all three models

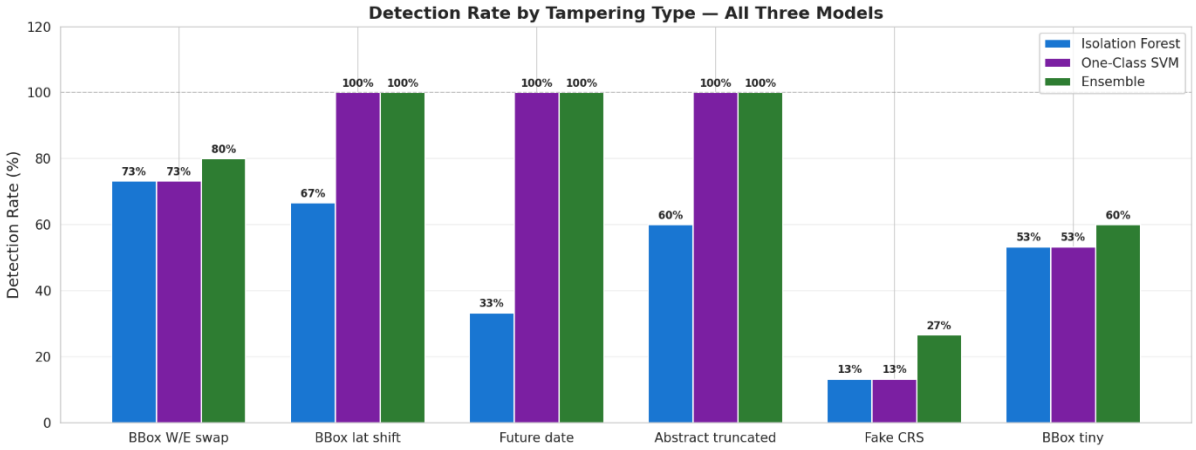


Figure 25: Detection rate of all three models

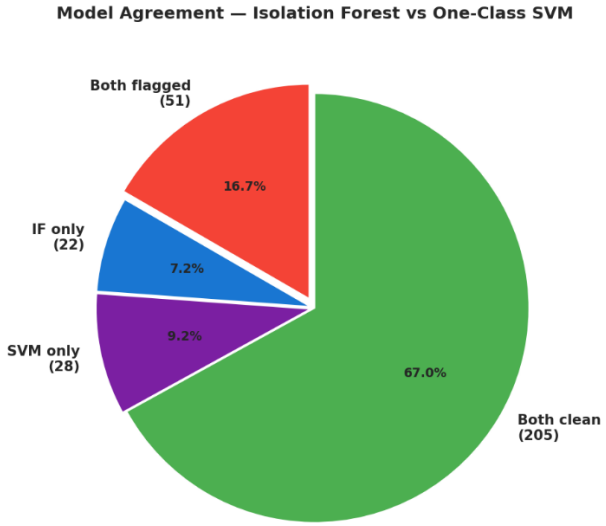


Figure 26: Model comparison pie chart in tampering detection in ML detector

Discussion

We have seen how the metadata tampering can impact GIS analysis, and finally developed a solution for the detection of metadata tampering in geospatial layers, in the QGIS. The plugin is not the perfect one in itself, but serves its major purpose.

Each component targets a distinct threat model and operational context: hash verification provides absolute certainty for trusted baselines, semantic checking provides baseline-free consistency validation, and ML detection surfaces sophisticated or previously unseen tampering patterns. Together, they form a robust, practical, and extensible integrity verification system that can be deployed across government, commercial, academic, and legal GIS environments.

Aspect	Hash + PBKDF2	Semantic Checker	ML Detector
Concept	Cryptography	Rule + NLP	Machine Learning
Input need	Original hash/db	Only metadata	Training data
Output	Exact (match/mismatch)	Logical consistency	Probability (suspicious)
Accuracy	100% (deterministic)	Moderate	Depends on training
Detects	Any change	Logical errors	Complex/hidden patterns
Explainability	High	High	Low
Flexibility	Low	Medium	High
Dependency	Password	Rules	Data + model

Figure 27: Three pillar verification model comparison in different aspects

Current Limitations –

- The ML Detector's accuracy is constrained by the size and diversity of the training dataset. Performance on novel tampering patterns not represented in training data may be lower than on known patterns.
- The Hash Verifier requires secure management of the original password. Loss of the password prevents hash-based verification, though the other two components remain functional.
- The Semantic Checker's rules are currently calibrated for standard ISO 19115 metadata structures. Highly non-standard or proprietary metadata schemas may require custom rule extensions.
- The plugin currently operates on single metadata files. Batch verification of large dataset collections has not been optimized.
- The cryptographic approach depends on password strength, making it vulnerable to brute-force attacks if weak passwords are used, despite PBKDF2 hardening.

Conclusion

This project presents a multi-layered approach for ensuring GIS metadata integrity by integrating cryptographic verification, semantic analysis, and machine learning–based anomaly detection within a QGIS plugin. Unlike traditional validation tools such as Geospatial Analysis Integrity Tool, which focus primarily on schema compliance, the proposed system addresses the critical gap of forensic metadata tampering detection.

The tool has strong applications across domains, including Defense GIS (secure geospatial intelligence), land records (protecting ownership data), flood mapping (ensuring reliable analysis), infrastructure planning (maintaining data trust), and research integrity (supporting reproducibility). Overall, it provides a practical and extensible approach to enhancing trust in geospatial metadata.

Despite certain limitations, this project serves as a strong foundation for approaching metadata security from a more critical and systematic perspective, encouraging the development of more robust, multi-layered solutions for ensuring trust in geospatial data.

Future Work –

- Integration with distributed ledger technology (blockchain) to provide decentralized, tamper-evident hash storage eliminating the single-point-of-failure of centralized hash databases.
- Extension of the ML Detector to support online/incremental learning, allowing the model to adapt to new tampering patterns identified in production without full retraining.
- Development of a server-side verification API to enable automated metadata checking in CI/CD pipelines for geospatial data platforms.
- Expanding the Semantic Checker with additional domain-specific rules for meteorological, hydrological, and urban planning metadata schemas.
- The NLP component can be improved using context-aware language models to better handle diverse metadata formats and reduce rule dependency.
- Natural language generation for human-readable integrity reports suitable for non-technical stakeholders.
- Further work may also include extending the tool to support spatial data integrity verification, cross-platform compatibility beyond QGIS, and integration with cloud-based GIS systems for large-scale deployments.

Contribution

- Ankit Kumar:
    - Pillar 3: Machine Learning Detector Development
    - Machine Learning Anomaly Detection Model Development
    - Metadata Feature Engineering (19 Numeric Features)
    - Ensemble Modeling with Isolation Forest and One-Class SVM
    - Performance evaluation of Model on Test data

    - Dataset Creation of 21 vector layers metadata from Natural earth
    - Results Analysis and Multi-Pillar Model Comparison
    - Current Limitations and Future Work Research
- Kanika Sharma:
    - GIS Analysis
    - Pillar 1: Cryptographic Model Development
    - Cryptographic Engine Implementation (HMAC-SHA256 & PBKDF2)
    - Metadata Value Normalization Logic Development
    - Secure Key Derivation and Salt Management
    - Combined Hash Construction for Integrity Fingerprinting
    - Automated Performance Evaluation and Metric Computation
- Shivesh Shukla:
    - NLP-Based Semantic Checker and 14-Rule Engine Design
    - Text Tokenization and Domain-Specific Stopword Removal
    - Vocabulary Assessment using TF-IDF and Shannon Entropy
    - Geometry-to-Vocabulary Domain MappingNLP based Semantic Checker model development and evaluation
    - Automated Performance Evaluation and Metric Computation
    - Metadata Verifier QGIS Plugin Architecture Design
    - Multi-threaded Background Processing for QGIS UI Responsiveness

References

[1] GIS metadata standard (ISO 19115) ISO, "ISO 19115:2003 Geographic Information – Metadata," International Organization for Standardization, Geneva, Switzerland, 2003. [Online]. Available: https://www.iso.org/standard/26020.html

[2] SHA-256 standard (NIST FIPS 180-4) National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS PUB 180-4, U.S. Department of Commerce, Aug. 2015. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/180/4/final

[3] SHA-256 RFC (formal technical reference) D. Eastlake and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," RFC 6234, IETF, May 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6234

[4] National Institute of Standards and Technology (NIST). Recommendation for Password-Based Key Derivation — Part 1: Storage Applications. NIST Special Publication 800-132. U.S. Department of Commerce, 2010. https://doi.org/10.6028/NIST.SP.800-132

[5] GIS security research paper (Bertino & Thuraisingham) E. Bertino and B. Thuraisingham, "Security and Privacy for Geospatial Data: Concepts and Research Directions," in Proc. ACM SIGSPATIAL Int. Workshop on Security and Privacy in GIS and LBS, 2008.

[6] Geospatial data integrity and security (recent, 2025) R. Marri et al., "Revolutionizing Geospatial Data Integrity and Sharing," Distributed Learning and Broad Applications in Scientific Research, vol. 9, pp. 490–511, 2024.

[7] QGIS documentation QGIS Development Team, "QGIS User Guide," QGIS Project, 2024. [Online]. Available: https://docs.qgis.org/latest/en/docs/user_manual/

[8] QGIS Plugin Development QGIS Development Team, "PyQGIS Developer Cookbook," QGIS Project, 2024. [Online]. Available: https://docs.qgis.org/latest/en/docs/pyqgis_developer_cookbook/
[9] Spatial data integrity research A. Worboys and M. Duckham, GIS: A Computing Perspective, 2nd ed. Boca Raton, FL: CRC Press, 2004.
[10] QGIS OpenStreetMap QGIS OpenStreetMap: OSM Plugins for QGIS - GIS Geography
[11] Natural Earth Data Natural Earth » Downloads - Free vector and raster map data at 1:10m, 1:50m, and 1:110m scales
[12] Copernicus Marine Service (CMEMS). Copernicus Marine Environment Monitoring Service — Open Access Metadata Catalogue. European Commission, 2024. https://marine.copernicus.eu
[13] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0
[14] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), Article 15. https://doi.org/10.1145/1541880.1541882